

Assignment 4

*Shared Memory with Multiple Blocks & Multiple Threads:
Matrix Multiplication*

Name: Arnav Singhal

MIS: 612203012

Div: 1; Batch: B3

1 Introduction

This assignment implements matrix multiplication using CUDA shared memory, distributed across multiple blocks and multiple threads per block, in order to understand how shared memory reduces redundant global memory traffic in a tiled computation. Two 512×512 integer matrices are multiplied using a tiled shared-memory kernel with a tile width of 16, giving a 32×32 grid of 16×16 -thread blocks. To quantify the benefit, an unparallelized CPU version of the same multiplication is also implemented, and the execution time of both versions is measured and compared. The program was compiled with `nvcc -arch=sm_75` and executed on a Google Colab T4 GPU runtime.

2 The Program: Matrix Multiplication with Timing Comparison

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define N 512
#define TILE_WIDTH 16

__global__ void matMulShared(int A[][N], int B[][N], int C[][N], int n) {
    __shared__ int tileA[TILE_WIDTH][TILE_WIDTH];
    __shared__ int tileB[TILE_WIDTH][TILE_WIDTH];

    int row = blockIdx.y * TILE_WIDTH + threadIdx.y;
    int col = blockIdx.x * TILE_WIDTH + threadIdx.x;

    int sum = 0;

    for (int t = 0; t < n / TILE_WIDTH; t++) {
        tileA[threadIdx.y][threadIdx.x] = A[row][t * TILE_WIDTH + threadIdx.x];
        tileB[threadIdx.y][threadIdx.x] = B[t * TILE_WIDTH + threadIdx.y][col];

        __syncthreads();

        for (int k = 0; k < TILE_WIDTH; k++) {
            sum += tileA[threadIdx.y][k] * tileB[k][threadIdx.x];
        }

        __syncthreads();
    }

    C[row][col] = sum;
}

void matMulCPU(int A[][N], int B[][N], int C[][N], int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            int sum = 0;
            for (int k = 0; k < n; k++) {
                sum += A[i][k] * B[k][j];
            }
            C[i][j] = sum;
        }
    }
}
```

```

void random_ints(int a[][N], int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            a[i][j] = (rand() % 10) + 1;
        }
    }
}

void print_corner(int a[][N], const char *label) {
    printf("%s\n", label);
    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < 5; j++) printf("%d ", a[i][j]);
        printf("\n");
    }
    printf("\n");
}

int main(void) {
    int (*A)[N], (*B)[N], (*C_cpu)[N], (*C_gpu)[N];
    int (*d_A)[N], (*d_B)[N], (*d_C)[N];
    int size = N * N * sizeof(int);

    srand(time(NULL));

    A = (int (*)[N])malloc(size); random_ints(A, N);
    B = (int (*)[N])malloc(size); random_ints(B, N);
    C_cpu = (int (*)[N])malloc(size);
    C_gpu = (int (*)[N])malloc(size);

    clock_t cpu_start = clock();
    matMulCPU(A, B, C_cpu, N);
    clock_t cpu_end = clock();
    double cpu_time_ms = ((double)(cpu_end - cpu_start) / CLOCKS_PER_SEC) * 1000.0;

    cudaMalloc((void **)&d_A, size);
    cudaMalloc((void **)&d_B, size);
    cudaMalloc((void **)&d_C, size);

    cudaMemcpy(d_A, A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, B, size, cudaMemcpyHostToDevice);

    dim3 dimBlock(TILE_WIDTH, TILE_WIDTH);
    dim3 dimGrid(N / TILE_WIDTH, N / TILE_WIDTH);

    cudaEvent_t gpu_start, gpu_stop;
    cudaEventCreate(&gpu_start);
    cudaEventCreate(&gpu_stop);

    cudaEventRecord(gpu_start);
    matMulShared<<<dimGrid, dimBlock>>>(d_A, d_B, d_C, N);
    cudaEventRecord(gpu_stop);
    cudaEventSynchronize(gpu_stop);

    float gpu_time_ms = 0;
    cudaEventElapsedTime(&gpu_time_ms, gpu_start, gpu_stop);

    cudaMemcpy(C_gpu, d_C, size, cudaMemcpyDeviceToHost);

```

```

printf("Matrix size: %d x %d\n", N, N);
printf("Tile size: %d x %d\n", TILE_WIDTH, TILE_WIDTH);
printf("Threads per block: %d x %d (%d total)\n", dimBlock.x, dimBlock.y,
    dimBlock.x * dimBlock.y);
printf("Grid dimensions: %d x %d\n", dimGrid.x, dimGrid.y);
printf("Total blocks: %d\n\n", dimGrid.x * dimGrid.y);

print_corner(A, "Top-left 5x5 of A:");
print_corner(B, "Top-left 5x5 of B:");
print_corner(C_cpu, "Top-left 5x5 of C (CPU result):");
print_corner(C_gpu, "Top-left 5x5 of C (GPU result):");

printf("Unparallelized (CPU) execution time: %f ms\n", cpu_time_ms);
printf("Shared memory (GPU) execution time: %f ms\n", gpu_time_ms);
printf("Speedup (unenhanced / enhanced): %f\n", cpu_time_ms / gpu_time_ms);

free(A); free(B); free(C_cpu); free(C_gpu);
cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
cudaEventDestroy(gpu_start);
cudaEventDestroy(gpu_stop);

return 0;
}

```

```
!nvcc -arch=sm_75 matmul_compare.cu -o matmul_compare

!./matmul_compare

Matrix size: 512 x 512
Tile size: 16 x 16
Threads per block: 16 x 16 (256 total)
Grid dimensions: 32 x 32
Total blocks: 1024

Top-left 5x5 of A:
1 10 3 9 9
9 10 5 9 6
8 10 2 7 9
2 4 7 9 6
9 9 6 4 6

Top-left 5x5 of B:
6 5 4 10 8
10 1 6 3 6
5 7 7 8 1
2 9 8 5 4
1 4 6 3 2

Top-left 5x5 of C (CPU result):
14762 15533 15561 15211 14652
15727 16291 16354 15965 15626
14969 15692 15867 15720 14956
15287 15518 15611 15297 15180
15208 15559 15420 15522 14778

Top-left 5x5 of C (GPU result):
14762 15533 15561 15211 14652
15727 16291 16354 15965 15626
14969 15692 15867 15720 14956
15287 15518 15611 15297 15180
15208 15559 15420 15522 14778

Unparallelized (CPU) execution time: 1053.661000 ms
Shared memory (GPU) execution time: 0.701760 ms
Speedup (unenhanced / enhanced): 1501.454926
```

Figure 1: Output showing the matrix corners, the CPU and GPU execution times, and the computed speedup.

3 Shared Memory, Blocks & Threads

- `__shared__ int tileA[TILE_WIDTH][TILE_WIDTH], tileB[...]`: on-chip shared memory arrays, allocated per block and visible to every thread in that block, used to stage one tile of A and B so the tile's data is read from global memory only once instead of once per thread.
- `dim3 dimBlock(TILE_WIDTH, TILE_WIDTH)`: a 2D block of $16 \times 16 = 256$ threads, matching the tile dimensions so each thread loads exactly one element per tile.
- `dim3 dimGrid(N / TILE_WIDTH, N / TILE_WIDTH)`: a 2D grid of 32×32 blocks, together covering the full 512×512 output matrix.
- `row, col`: computed from `blockIdx` and `threadIdx` together, giving each thread a unique output element to compute, consistent with the global-indexing pattern used in the vector

addition assignment.

- `__syncthreads()`: barrier that ensures every thread in the block has finished loading its tile element before any thread starts reading from shared memory, and again before the next tile is loaded, preventing race conditions on the shared arrays.

4 Measuring Execution Time

The CPU version is timed with the standard C `clock()` function, taking a reading before and after the call to `matMulCPU` and converting the difference to milliseconds using `CLOCKS_PER_SEC`. The GPU version is timed using CUDA's event API: `cudaEventCreate` allocates two event objects, `cudaEventRecord` marks the start and end of the kernel launch on the GPU's own timeline, `cudaEventSynchronize` blocks the host until the stop event has actually been reached on the device, and `cudaEventElapsedTime` returns the elapsed time between the two events in milliseconds. CUDA events are used instead of a host-side clock for the GPU because kernel launches are asynchronous, so only device-side timestamps give an accurate measurement of the kernel's own execution time. The speedup is then computed as the CPU time divided by the GPU time.

5 Conclusion

The shared-memory tiled kernel and the plain triple-loop CPU implementation were shown to produce matching results, confirming the correctness of the tiling and indexing scheme. Timing both versions, the CPU implementation using `clock()` and the GPU implementation using CUDA events, allowed a direct speedup comparison, demonstrating the performance benefit of distributing the multiplication across multiple blocks and threads while using shared memory to minimize redundant global memory accesses.